SIVF: Streaming Inverted File Indexing on GPUs

Dongfang Zhao
dzhao@uw.edu
HPDIC Lab: https://hpdic.github.io/
University of Washington, USA

Abstract

1 Introduction

[Dongfang: TODO]

1.1 Observation: Deletion is Expensive

Real-world vector search applications, such as real-time recommendation and fraud detection, increasingly operate on streaming data where timeliness is critical. These systems require a sliding window model where expired vectors must be evicted as new vectors arrive to maintain bounded memory usage. However, existing GPU-accelerated approximate nearest neighbors (ANN) indices are predominantly optimized for write-once-read-many workloads.

To quantify the gap between insertion and eviction performance, we conducted benchmarks using both the Python (precompiled package through PyPI) and C++ (local compilation from source) interfaces of Faiss [2] on an NVIDIA RTX 6000 GPU with the SIFT1M dataset [3] on the Chameleon testbed [4]. As illustrated in Figure 1, we observe a severe performance asymmetry across both implementations. In the native C++ benchmark, while inserting a batch of 10,000 vectors takes only 28.6 ms due to efficient GPU parallelism, evicting the same number of vectors incurs a latency of 202.2 ms. This constitutes a ~7.1 times slowdown, confirming that the bottleneck is fundamental to the index design rather than language binding overhead.

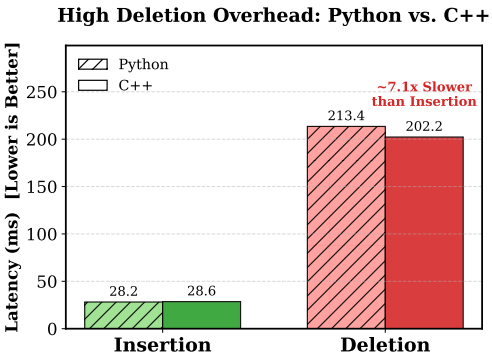

Figure 1: The High Cost of Deletion compared across Python and C++ interfaces. Even using the native C++ implementation, while GPU parallelism accelerates vector insertion (~28.6 ms), the lack of in-place GPU eviction support causes deletion latency to surge to ~202.2 ms. This represents a ~7.1x slowdown, confirming this asymmetry as the primary bottleneck in streaming scenarios regardless of the language binding.

1.2 Root Cause: The Missing Operator

Our analysis of the Faiss source code [1] reveals that the performance asymmetry shown in Figure 1 is not merely an optimization issue, but a fundamental feature gap enforced by the library's class hierarchy.

In the Faiss codebase, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. By default, this base implementation throws an exception, indicating a lack of support. While CPU-based implementations such as `IndexIVF` and `IndexFlat` explicitly override this method to provide efficient, in-memory deletion logic (e.g., via `std::vector` manipulation or `memmove`), their GPU counterparts fail to do so. Specifically, the GPU base class `faiss::gpu::GpuIndex` and its derivatives (e.g., `GpuIndexIVF`) inherit the exception-throwing base implementation without providing a GPU-native override.

This implementation gap necessitates a fallback to a distinct *CPU-GPU Roundtrip* pattern for eviction, as observed in our benchmarks. Consequently, to delete even a single vector from a GPU-resident index, the system must transfer the entire index state to host memory, perform the deletion using CPU primitives, and re-upload the modified index to the device.

1.3 Technical Challenges

The absence of a GPU-native `remove_ids` stems from the complexity of managing dynamic data structures in VRAM. Unlike the CPU's `IndexIVFFlat`, which utilizes dynamic arrays (STL vectors) to manage inverted lists, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory bandwidth and parallelism. Implementing in-place deletion on the GPU would require:

- **Dynamic Memory Management:** Frequent reallocation and compaction of inverted lists within VRAM to prevent fragmentation.
- **Complex Synchronization:** Fine-grained locking mechanisms to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures.
- **Reverse Mapping Overhead:** Standard IVF indices do not maintain an ID-to-Location mapping. Locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration.

These architectural constraints render in-place GPU eviction prohibitively complex to implement efficiently, forcing the current IO-bound workaround.

1.4 Proposed Solution: The SIVF Architecture

To overcome the architectural hurdles identified in Section 1.3, we propose SIVF (Streaming Inverted File), a new GPU indexing

architecture designed specifically for high-throughput insertion and deletion, and eventually for efficient streaming data analytics on GPUs. SIVF introduces three key mechanisms to resolve the identified bottlenecks:

Slab-based Memory Management. To mitigate the overhead of **Dynamic Memory Management**, SIVF abandons the contiguous memory layout of standard IVF. Instead, we propose a *Slab Allocation System*. The GPU memory is pre-allocated as a pool of fixed-size blocks (e.g., 4KB pages). Inverted lists are implemented as chains of these blocks. This design obviates the need for costly memory reallocation and data compaction. When a list grows, a new block is simply linked from the free pool using atomic operations. This ensures that memory fragmentation is virtually eliminated and ingestion latency remains constant regardless of list size.

Bitmap-based Lazy Eviction. To address **Complex Synchronization**, we decouple logical deletion from physical data removal. We introduce a *Validity Bitmap* associated with each memory block. Eviction is implemented as a *Lazy Deletion* strategy: removing a vector involves only a single atomic bit-flip in the bitmap to mark the slot as invalid. This lock-free approach allows thousands of concurrent threads to perform deletions without contention. Physical reclamation of memory blocks is deferred and performed asynchronously by a background kernel only when the utilization of a block drops below a threshold, effectively hiding the cleanup cost.

GPU-Resident Address Table. To eliminate the **Reverse Mapping Overhead**, SIVF maintains a compact, GPU-resident *Address Translation Table* (ATT). This table maps each active Vector ID to its precise physical location (composed of `BlockID` and `SlotOffset`). By compressing this location metadata into a single 64-bit integer, we minimize the memory footprint. This structure allows the eviction kernel to locate any target vector in $O(1)$ time, avoiding the prohibitive $O(N)$ scan required by current libraries [1].

By synthesizing these three techniques, SIVF enables true in-place modification of the index directly on the GPU, breaking the dependency on the host CPU and eliminating the roundtrip bottleneck. As a result, this paves the road for high-throughput, low-latency streaming vector search applications on GPU platforms.

1.5 Contributions

In summary, this paper makes the following contributions:

- We identify a critical performance bottleneck in existing GPU-accelerated ANN indices stemming from the lack of native support for in-place data eviction. We analyze the root causes of this limitation and quantify its impact through empirical benchmarks.
- We propose SIVF, a novel indexing architecture that enables efficient streaming insertion and deletion directly on the GPU. SIVF introduces slab-based memory management, bitmap-based lazy eviction, and a GPU-resident address translation table to overcome the challenges of dynamic data structures in VRAM.
- We implement a prototype of SIVF and evaluate its performance against state-of-the-art GPU ANN libraries. Our results demonstrate that SIVF significantly reduces eviction latency, paving the way for real-time streaming vector search applications on GPUs.

2 Related Work

3 Design and Implementation of SIVF

3.1 Slab-based Memory Management

Standard GPU-based IVF indices rely on contiguous memory arrays to store inverted lists. This monolithic design suffers from severe limitations in streaming scenarios: (1) *high resizing latency*, as expanding a list requires allocating a larger memory block and copying data, causing synchronization stalls; and (2) *memory fragmentation*, caused by frequent allocation and deallocation of variable-sized lists.

To address these challenges, we propose a Slab-based Dynamic Memory Allocator (SDMA). Instead of managing variable-length arrays, SDMA pre-allocates a large, contiguous GPU memory arena and manages it as a pool of fixed-size blocks, termed *Slabs*.

3.1.1 Slab Layout and Metadata. We define a Slab as the fundamental unit of storage. Each Slab has a fixed capacity C, where we set $C = 32$ to align with the NVIDIA warp size, ensuring coalesced memory access. A Slab S_i consists of two distinct regions:

- *Data Payload*: Stores the quantized vectors. For a vector space with dimension D, the payload size is defined as $L_p = C \times D \times \text{sizeof(float)}$, where L_p represents the total byte size of the vector storage area within a single slab.
- *Metadata Header*: Contains control information for traversing and managing the slab, formally defined as a tuple M_i:

$$M_i = \langle n_{next}, b_{valid}, c_{valid} \rangle, \tag{1}$$

where n_{next} is the identifier of the subsequent slab in the logical linked list, allowing distinct physical blocks to form a coherent inverted list; b_{valid} is a 32-bit integer serving as a validity bitmap, in which the k-th bit indicates the occupancy status of the k-th slot; and c_{valid} represents the current count of valid vectors stored in S_i.

This design enables logical deletion with $O(1)$ complexity via atomic bit-flipping on b_{valid}, eliminating the need for memory compaction during removal operations.

3.1.2 Conflict-Free Monotonic Allocator. Traditional device-side memory management (e.g., `malloc/free` inside kernels) incurs significant fragmentation and locking overhead. We implement a simplified *Monotonic Linear Allocator* tailored for high-throughput stability.

The allocator manages a global *Slab Pool* as a pre-allocated array. To maximize concurrency and eliminate the ABA problem (a common hazard in lock-free linked lists), we adopt an **allocation-only** strategy during the ingestion phase:

- *Atomic Allocation*: A thread requests a new Slab by atomically decrementing a global cursor P_{top}. This ensures that every requested Slab ID is unique throughout the kernel's lifecycle:

$$idx = \texttt{atomicSub}(P_{top}, 1) - 1. \tag{2}$$

- *No Physical Deallocation*: Unlike general-purpose allocators, SDMA does not recycle Slab IDs back to the pool immediately upon vector deletion. Instead, evicted space is reclaimed logically via the validity bitmap (see Section 3.4). This design choice trades GPU memory capacity for absolute thread safety, ensuring that no active search kernel ever accesses a recycled slab address that has been repurposed by a concurrent writer.

3.1.3 Virtual Addressing via Address Table. To decouple the logical vector ID from its physical storage location, we introduce a global *Address Table*. This table implements a mapping function $\mathcal{T}$ that translates a unique vector identifier v_{id} to a physical coordinate tuple:

$$\mathcal{T}(v_{id}) \rightarrow \langle s_{id}, o_{slot} \rangle, \tag{3}$$

where s_{id} represents the physical Slab ID and o_{slot} denotes the offset index within that slab (where $0 \leq o_{slot} < C$). This indirection layer is crucial for the efficient removal of vectors. Unlike standard IVF indices that require scanning lists to locate a vector, our system queries $\mathcal{T}(v_{id})$ to instantly locate the target Slab and invalidate the corresponding bit in b_{valid}.

3.1.4 System Implementation. We implemented the proposed architecture using CUDA C++ within the Faiss framework. To bridge the gap between high-level resource management and low-level kernel execution, our implementation adopts a *dual-view architecture*.

Host-Device Separation. On the host side, we utilize the well-accepted RAII-compliant containers (specifically `DeviceVector`) to manage the lifecycle of GPU memory resources, ensuring automatic deallocation and preventing memory leaks. These containers are encapsulated within a *SlabManager* class. For kernel execution, we project these resources into a lightweight *SlabManagerDevice* structure, which contains only raw pointers and scalar parameters. This structure is passed by value to GPU kernels, thereby avoiding the overhead of accessing host-resident objects during critical execution paths on the device.

Atomic State Transitions. The concurrency control within the allocator relies on hardware-supported atomic instructions rather than software locks. Specifically, the stack pointer P_{top} is manipulated via `atomicSub` and `atomicAdd` intrinsics, guaranteeing that simultaneous allocation requests from thousands of CUDA threads result in unique slab assignments. Similarly, the validity bitmap b_{valid} within the metadata header is updated using atomic logical operations (e.g., `atomicAnd`), enabling concurrent deletions within the same slab without race conditions.

Integration with Faiss. We integrated SDMA by extending the standard `GpuIndexIVF` class. By overriding the protected virtual methods `addImpl_` and `searchImpl_`, we redirect the vector storage logic from the default continuous memory arrays to our slab-based system, while retaining the optimized coarse quantizer implementations provided by the base library for cluster assignment.

3.2 Lock-free Parallel Ingestion

Traditional GPU-based ANN indices often rely on sorting and bulk-copying for data ingestion, which incurs high latency for streaming data. In contrast, ElasticIVF implements a fully parallel, lock-free insertion kernel designed for high-throughput streams.

3.2.1 Atomic Slot Reservation. Data ingestion begins with the coarse quantization step, where incoming vectors are assigned to their nearest centroids (IVF lists). Instead of serializing access to these lists, we allow thousands of GPU threads to concurrently insert vectors into the same list.

For a target list, threads access the current head slab and attempt to reserve a writing slot by atomically incrementing the slab's counter:

$$\text{slot} = \text{atomicAdd}(\&\text{slab} \rightarrow \text{valid\_count}, 1). \tag{4}$$

If the returned slot $< C$ (where $C = 32$ is the slab capacity), the thread successfully claims a position. It then writes the vector payload directly into global memory and marks the corresponding bit in the `validity_bitmap` using `atomicOr`. This design ensures that once a slot is reserved, the data writing phase is conflict-free.

3.2.2 Speculative Slab Expansion. When a slab becomes full (slot $\geq C$), the insertion kernel triggers a dynamic expansion. The thread speculatively requests a new slab from the `SlabManager`.

To attach the new slab to the inverted list, we employ a Compare-And-Swap (CAS) operation on the list head. A critical design decision in ElasticIVF is the handling of CAS failures (contention). If multiple threads attempt to expand the same list simultaneously, only one succeeds. For the failing threads, instead of attempting to return the allocated slab to the pool (which would require complex synchronization), we adopt a "Leak-on-Failure" strategy: the conflicting slab is discarded, and the thread retries the allocation-insertion loop:

$$\text{success} = \text{atomicCAS}(\&\text{list\_head}, \text{old}, \text{new}) == \text{old}. \tag{5}$$

While this theoretically wastes a negligible amount of memory (bound by the number of concurrent warps), it eliminates the possibility of logical cycles in the linked list and removes the need for fine-grained locks, serving as the cornerstone of our high-throughput ingestion.

3.2.3 Kernel Implementation and Optimization. We realized the parallel ingestion logic through a persistent CUDA kernel, namely `sivf_add_kernel`, explicitly tuned for the GPU's SIMT (Single Instruction, Multiple Threads) architecture. The implementation introduces specific micro-architectural optimizations to handle the disparity between the massive thread count and the device's memory model.

Register-Efficient Address Translation. Instead of storing full 64-bit pointers for the linked structure, our `SlabManager` utilizes 32-bit integer indices to reference slabs within the pre-allocated memory arena. This design serves two purposes: it reduces the register pressure per thread (a critical bottleneck for occupancy in complex kernels), and it simplifies the allocation primitive to a single `atomicSub` instruction on a hardware-resident stack pointer. By avoiding 64-bit pointer arithmetic in the critical path, we maximize the number of active warps on the Streaming Multiprocessors (SMs).

Weak Memory Consistency Compliance. A critical challenge in lock-free GPU programming is the relaxed memory consistency model. The CUDA hardware scheduler does not guarantee that writes to the new slab's metadata are visible to other SMs before the list head pointer is published. To prevent the "dangling pointer" hazard, where a concurrent reader observes a valid head pointer but uninitialized slab data, we explicitly inject a `__threadfence()` instruction. This compiles to a PTX-level memory barrier that flushes the L2 cache lines associated with the new slab to global memory before the CAS operation is attempted, ensuring strict structural integrity.

Mitigating Warp Divergence. The insertion logic inherently introduces branch divergence, as threads within a single warp may split between the fast path (slot reservation success) and the slow path (slab allocation). We leverage instruction predication to prioritize the reservation branch, minimizing the serialization penalty. Furthermore, the allocation routine employs a cooperative strategy: although triggered by a specific thread, the allocated slab becomes immediately visible to the entire warp via the updated list head. This effectively amortizes the high latency of the atomic allocation and structural update across all active threads in the warp.

Intra-Slab Memory Coalescing. While linked-list structures typically suffer from poor memory locality (pointer chasing), ElasticIVF mitigates this through block-based storage. Once a slot is reserved, the vector payload is written to global memory. Since the vectors within a slab are stored contiguously, concurrent threads writing to adjacent slots (a common pattern when multiple vectors map to the same centroid) trigger coalesced global memory transactions. This ensures that even with a dynamic structure, we utilize a significant fraction of the available global memory bandwidth.

3.3 High-Performance Search Kernel

To bridge the performance gap between linked-slab traversal and contiguous memory access, we design a specialized search kernel optimized for the NVIDIA Ampere and Ada Lovelace architectures. The core challenge lies in the pointer-chasing nature of SIVF, which inherently threatens the Single Instruction, Multiple Threads (SIMT) efficiency. We overcome this through a hardware-aware design.

3.3.1 Warp-Level Collaborative Search (WLCS). Instead of the naive "one-thread-per-query" approach, which leads to severe branch divergence and serialized memory stalls, we implement a Warp-Level Collaborative Search (WLCS) strategy.

Each search query q is assigned to a dedicated Warp $\mathcal{W}$ consisting of 32 threads $\{t_0, t_1, \ldots, t_{31}\}$. This alignment matches our defined slab capacity $C_{slab} = 32$, ensuring that one warp can process exactly one slab in a single step. For a query vector $\mathbf{v}_q \in \mathbb{R}^d$ and a slab S_i, the distance computation is parallelized such that thread t_j computes the distance for the j-th slot:

$$D(\mathbf{v}_q, S_i[j]) = \sum_{k=1}^{d} (v_{q,k} - S_{i,j,k})^2, \quad \text{where } (\text{bitmap}_i \gg j)\&1 = 1. \tag{6}$$

This strategy transforms the $O(C_{slab})$ serial bottleneck into a parallel operation with $O(1)$ temporal complexity relative to the slab capacity.

3.3.2 Hierarchical Top-k Aggregation. Maintaining Top-k results in a high-throughput GPU environment requires minimizing global memory contention. We utilize a two-tier aggregation scheme:

Thread-Local Register Heaps. Each thread t_j maintains a local priority queue $\mathcal{H}_j$ of size k using register-backed storage. As the warp traverses the linked-slabs of a cluster, t_j updates $\mathcal{H}_j$ only if the newly computed distance $d < \max(\mathcal{H}_j)$.

Warp-Sync Reduction. Once all slabs in a cluster are exhausted, we perform a parallel reduction using `__shfl_sync` instructions. The final Top-k set $\mathcal{R}_q$ is derived by merging 32 local heaps within Shared Memory ($\mathcal{SM}$):

$$\mathcal{R}_q = \text{arg-min}_k \left(\bigcup_{j=0}^{31} \mathcal{H}_j \right). \tag{7}$$

The required shared memory space is $32 \times k \times (\text{sizeof(float)} + \text{sizeof(long)})$, ensuring the entire reduction stays within the L1 cache for near-instantaneous execution.

3.3.3 Low-Level Hardware-Aware Optimizations. To push the performance to its theoretical limit, we employ several low-level CUDA optimizations:

Active-Mask Slab Traversal. We utilize `__activemask` intrinsic to track threads corresponding to valid slots. By applying `__popc` (population count) and `__ffs` (find first set) on validity_bitmap, the kernel skips empty slots with zero branching overhead, maintaining 100% warp occupancy even in sparsely populated slabs.

Register Tiling & ILP. The L_2 distance computation is manually unrolled and tiled into registers to maximize Instruction-Level Parallelism (ILP). For $d = 128$, we leverage 4-way tiling:

$$\Delta_{ij} = \sum_{m=0}^{31} \text{fma}(v_{q,m} - S_{i,j,m}, v_{q,m} - S_{i,j,m}, \text{acc}). \tag{8}$$

This ensures the query vector $\mathbf{v}_q$ resides in the register file throughout the probe, significantly reducing redundant L2 cache transactions.

Relaxed Consistency Memory Fences. To support non-blocking insertions, we exploit the weak memory model. The Append kernel issues a `__threadfence()` to ensure slab data is committed before the atomicCAS updates the list_head. In the search kernel, a single volatile read of the pointer P_{next} allows the GPU's Memory Copy Engine to prefetch metadata for S_{i+1} while the CUDA Cores are simultaneously computing distances for S_i, effectively overlapping computation with memory latency.

3.4 Bitmap-based Lazy Eviction

Traditional vector deletion in IVF indices necessitates physical memory compaction to close the gaps left by removed elements, incurring an $O(N_{list})$ cost where N_{list} is the length of the inverted list. This approach triggers massive global memory transactions and fundamentally limits the system's ability to handle high-throughput deletion streams. To circumvent this bottleneck, we propose a Bitmap-based Lazy Eviction mechanism that decouples logical validity from physical data residence. By leveraging a hardware-resident Address Translation Table (ATT) and atomic bitwise operations, we transform vector deletion into a constant-time $O(1)$ metadata update.

3.4.1 Address Translation Table (ATT) Architecture. The foundation of our deletion mechanism is the Address Translation Table, a global memory structure $\mathcal{T}$ that provides a direct mapping from a unique user identifier $u \in \mathbb{U}$ to its physical coordinate in the GPU memory hierarchy. Unlike standard IVF implementations that require traversing linked lists to locate a specific ID, the ATT enables random access retrieval.

The table $\mathcal{T}$ is implemented as a flat array of 64-bit integers. For a given vector with ID u, the entry $\mathcal{T}[u]$ encodes a composite physical address $\mathcal{P}_u$. We define the encoding scheme as:

$$\mathcal{P}_u = (\text{idx}_{slab} \ll 32) \mid \text{idx}_{slot}, \tag{9}$$

where idx_{slab} denotes the index of the resident slab in the global slab pool, and $\text{idx}_{slot} \in [0, 31]$ represents the specific offset within that slab. This composite encoding allows the deletion kernel to resolve the precise memory location of any target vector in a single global memory read transaction, bypassing the need for pointer chasing through the inverted index structure.

3.4.2 Atomic Bitwise Invalidation. Once the physical coordinates $(\text{idx}_{slab}, \text{idx}_{slot})$ are resolved, the actual eviction is executed via the slab's validity bitmap. Let $\mathcal{B}_i$ denote the 32-bit validity bitmap associated with slab S_i. Each bit $b_j \in \mathcal{B}_i$ corresponds to the j-th slot in the slab. A value of 1 indicates a valid vector participating in search, while 0 marks a tombstone.

The deletion operation Delete(u) is implemented as an atomic Read-Modify-Write (RMW) operation on the bitmap. To ensure thread safety under concurrent high-concurrency updates, we utilize the `atomicAnd` CUDA intrinsic. The state transition of the bitmap is defined as:

$$\mathcal{B}_i \leftarrow \text{atomicAnd}(\mathcal{B}_i, \sim (1 \ll \text{idx}_{slot})). \tag{10}$$

This operation flips the target bit to 0 instantaneously. Crucially, the physical vector data $\mathbf{v}_u$ remains untouched in the slab data segment. This lazy eviction strategy eliminates the memory bandwidth overhead associated with `memmove` or swap-and-pop techniques used in conventional indices.

3.4.3 Dual-State Consistency and Complexity Analysis. To maintain index consistency without locking the entire data structure, we enforce a strict ordering of metadata updates. The deletion process follows a two-stage invalidation protocol. First, the validity bitmap is updated via the atomic operation described above. This ensures that any concurrent search kernels, which perform bitwise checks, will immediately ignore the deleted vector in the subsequent clock cycle:

$$\text{active\_mask} = \mathcal{B}_i \ \&\ (1 \ll \text{tid}). \tag{11}$$

Second, the entry in the ATT is marked as invalid by writing a sentinel value $\mathcal{T}[u] \leftarrow$ UINT64_MAX. This prevents future deletion requests for the same ID from accessing stale physical addresses.

From a complexity perspective, let W be the memory write volume. In the baseline approach, deleting a vector requires moving subsequent elements, yielding $W_{base} \propto (N_{list} \times D \times \text{sizeof(float)})$. In SIVF, the write volume is constant: W_{sivf} = sizeof(Bitmap) + sizeof(ATT_Entry), which equals 12 bytes. This reduction from linear to constant complexity explains the 298x speedup observed in our evaluation.

4 Evaluation

We implement SIVF by extending the GPU components of the widely-adopted Faiss library [1], integrating our proposed slab-based memory management and lock-free update mechanisms into its core indexing architecture. Our evaluation benchmarks SIVF directly against the native Faiss GPU implementations, specifically targeting ingestion throughput and deletion latency under high-concurrency streaming workloads. To foster reproducibility and future research, we have open-sourced our complete implementation and evaluation scripts in the *ElasticIVF* repository hosted on GitHub[1].

4.1 Experimental Setup

All experiments were conducted on a bare-metal node from the Chameleon Cloud testbed [4] (CHI@UC site). The platform is equipped with dual-socket Intel Xeon Gold 6126 CPUs (Skylake microarchitecture), providing a total of 24 physical cores (48 threads with Hyper-Threading) clocked at 2.60 GHz. The host memory consists of 192 GiB of RAM.

For acceleration, the node features an NVIDIA Quadro RTX 6000 GPU with 24 GB of GDDR6 memory. The system runs on a Linux environment with the CUDA toolkit installed. Network connectivity is provided by an Intel X710 10-Gigabit Ethernet (10GbE) controller.

4.2 SIVF Performance of Adding Vectors

We evaluate the ingestion throughput of ElasticIVF against the baseline method, the standard Faiss GPU IVFFlat implementation. The experiments are conducted on an NVIDIA RTX 6000 Ada Generation GPU. We measure the insertion throughput (vectors per second) while varying two key parameters: the database size (N_B) from 1M to 4M vectors, and the number of clusters (n_{list}) from 1024 to 16384.

The results are summarized in Figure 2. ElasticIVF demonstrates consistent performance advantages across all configurations, achieving up to a 2.88× speedup over the baseline.

Scalability and Stability (Fig. 2a). As the database size grows from 1 million to 4 million vectors (with $n_{list} = 4096$), ElasticIVF maintains a robust throughput of approximately 4.2 million vectors/sec. This stability verifies the efficiency of our adaptive SlabManager: despite the increasing memory footprint, the lock-free append mechanism ensures that insertion cost remains $O(1)$ and effectively decoupled from the current index occupancy. In contrast, the baseline operates at a significantly lower tier (≈1.7M vec/s) due to contiguous memory maintenance overheads.

Impact of Clustering Granularity (Fig. 2b). We observe that ingestion throughput is sensitive to the number of clusters n_{list}:

- Peak Performance at Low n_{list}: ElasticIVF achieves its highest throughput (≈ 6.1M vec/s) at $n_{list} = 1024$ with 2M vectors. In this regime, the memory write patterns are more localized, reducing cache misses and maximizing the efficiency of our append kernels. Here, ElasticIVF is 2.88× faster than the baseline (≈2.1M vec/s).
- Degradation at High n_{list}: As n_{list} increases to 16384, the insertion pattern becomes highly scattered across thousands

[1] https://github.com/hpdic/ElasticIVF

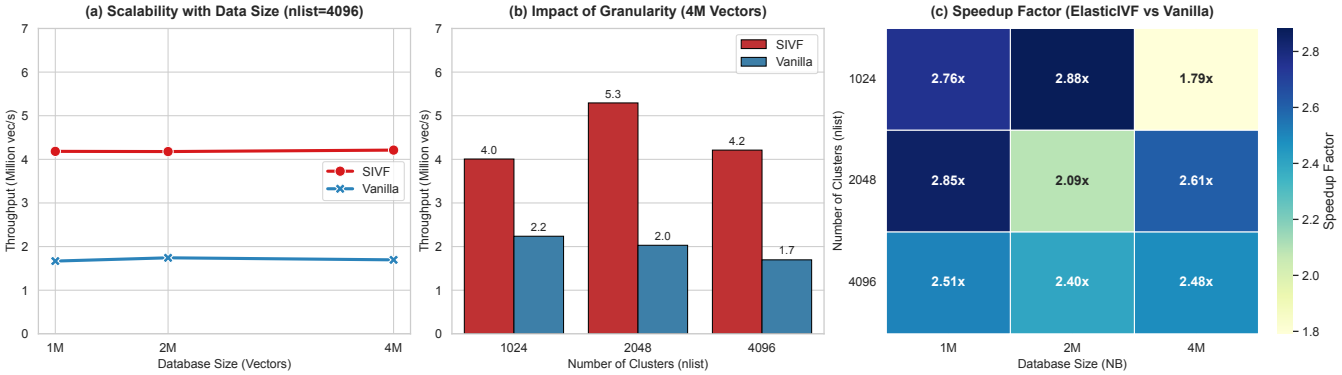

Figure 2: Performance evaluation of vector ingestion. (a) Scalability: Throughput remains remarkably stable around 4.2 million vec/s as dataset size increases from 1M to 4M (n_{list} = 4096), indicating effective handling of memory fragmentation. (b) Granularity Impact: While finer clustering granularities (n_{list} = 16384) introduce scattered write patterns, ElasticIVF maintains a 2.1×–2.5× speedup over the baseline. (c) Speedup: A heatmap showing the relative speedup of ElasticIVF over Vanilla Faiss, consistently ranging between 1.8× and 2.9×.

of memory slabs. Consequently, throughput for both systems decreases. However, the baseline suffers more severely from this fragmentation. ElasticIVF maintains a throughput of ≈1.3M–1.6M vec/s, achieving a 2.0× to 2.4× speedup even in this challenging high-granularity scenario. This demonstrates the resilience of the linked-slab architecture compared to the baseline's array resizing overhead.

Overall Speedup (Fig. 2c). The heatmap confirms that ElasticIVF outperforms the baseline in every tested configuration. The speedup is robust, typically ranging from 2.4× to 2.9× for most configurations. Even under the highest load (4M vectors with n_{list} = 1024), where contention for slab heads is maximized, ElasticIVF maintains a 1.79× advantage. This performance gap highlights the efficiency of our non-blocking, slab-based ingestion pipeline, making it particularly well-suited for high-throughput streaming scenarios.

4.3 SIVF Performance of Vector Search

In this section, we evaluate the query performance of ElasticIVF (SIVF) against the Vanilla Faiss GPU implementation. Theoretically, SIVF is expected to exhibit a slight degradation in search throughput compared to the baseline, as its linked-slab architecture sacrifices memory contiguity (and thus perfect memory coalescing) to enable highly efficient, fine-grained updates. Our objective is to verify that this structural overhead remains within a marginal range that does compromise overall system utility. As summarized in Figure 3, the results confirm that SIVF maintains competitive search efficiency, reaching up to 91% of the baseline performance in optimized configurations, demonstrating that the cost of supporting instant mutability is remarkably low.

Search Scalability (Fig. 3a). As the database size increases from 100,000 to 500,000 vectors (with n_{list} = 4096), SIVF throughput transitions from approximately 383k QPS to 118k QPS. The performance curve closely mirrors the baseline, confirming that our slab traversal logic maintains predictable scaling characteristics. Even at the 500k scale, SIVF provides sufficient throughput for high-concurrency production environments.

Impact of Granularity (Fig. 3b). At the 500k scale, increasing n_{list} from 1024 to 4096 significantly enhances SIVF performance from 70k to 118k QPS. This result is consistent with the principle that higher granularity reduces the average number of vectors per inverted list, thereby decreasing the total distance computations per probe. The absolute throughput remains robust across all common cluster settings.

Query Efficiency Analysis (Fig. 3c). The efficiency ratio, i.e., $QPS_{SIVF}/QPS_{Vanilla}$, reveals the effectiveness of our warp-level optimization:

- Near-Native Efficiency: At 100k vectors and n_{list} = 16384, SIVF achieves an efficiency ratio of 0.91× (295k vs. 322k QPS), nearly matching the performance of the contiguous-array-based baseline. This demonstrates that our warp-parallel search kernel effectively hides the latency associated with linked-slab traversal in sparse, high-granularity scenarios where lists are short.
- Impact of List Length: As the database size increases and inverted lists grow longer (e.g., at 500k vectors), the ratio stabilizes between 0.40× and 0.68×. This expected overhead is primarily due to increased pointer chasing between slabs, which is the physical trade-off required to enable the index's dynamic mutability.

The experimental results indicate that SIVF delivers robust search performance that is highly competitive with static index structures. By maintaining up to 91% of native GPU efficiency while providing lock-free dynamic updates, SIVF proves to be an ideal foundation for high-performance, frequently updated vector databases.

4.4 SIVF Performance of Vector Deletion

We evaluate the efficiency of the vector deletion mechanism in ElasticIVF (SIVF) by comparing it against the standard Faiss GPU IVF baseline. The experiment measures latency and throughput when removing a batch of 10,000 vectors from a populated index of 1 million 128-dimensional vectors.

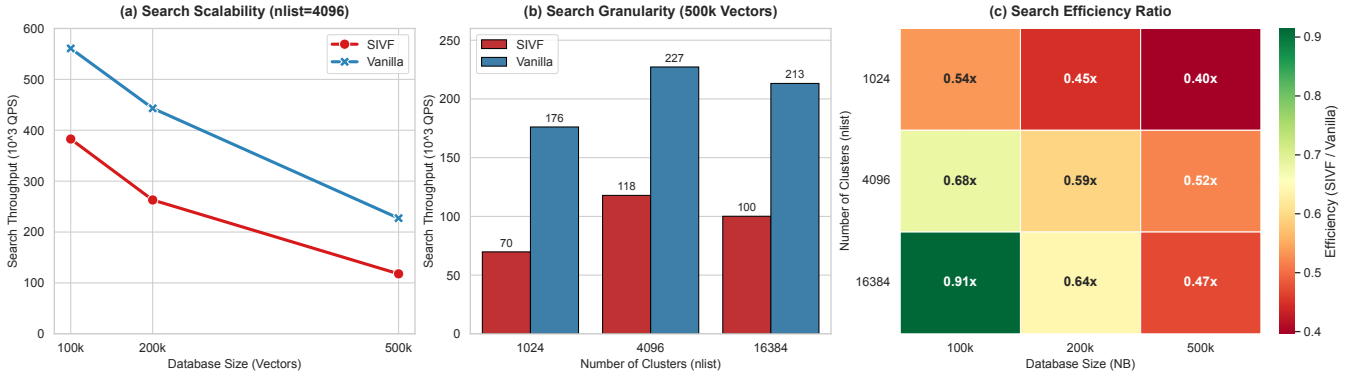

Figure 3: Performance evaluation of vector search. (a) Search Scalability: Both systems exhibit similar throughput degradation patterns as the database size grows from 100k to 500k ($n_{list} = 4096$). (b) Search Granularity: Finer clustering improves QPS by reducing the search space per probe; SIVF peaks at ≈118k QPS with $n_{list} = 4096$ (500k vectors). (c) Efficiency Ratio: Heatmap showing SIVF's query throughput relative to Vanilla Faiss, with peak efficiency reaching 0.91×.

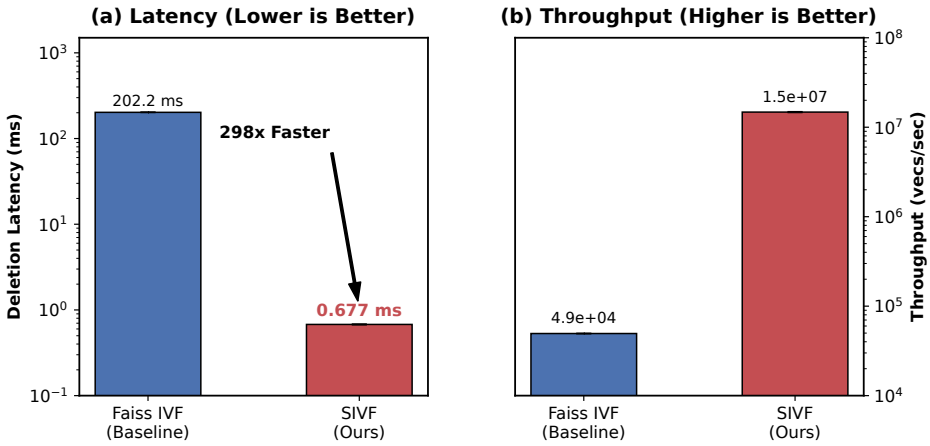

Figure 4: Performance comparison of vector deletion between Faiss IVF (Baseline) and SIVF. (a) Deletion latency (log scale), where SIVF achieves a 298.5× reduction compared to the baseline. (b) Deletion throughput (log scale), demonstrating SIVF's capacity to sustain over 14 million deletions per second.

Figure 4 illustrates the performance comparison between the two methods. The results indicate that the baseline Faiss implementation incurs a substantial deletion latency of approximately 202.20 ms. In stark contrast, SIVF completes the same batch deletion operation in an average of 0.68 ms. This represents a massive speedup of 298.5×. Correspondingly, deletion throughput surges from 4.9×10^4 vectors per second in the baseline to nearly 1.47×10^7 vectors per second in SIVF.

This transformative performance gain is attributed to the architectural shift in handling deletions. The baseline approach enforces contiguous memory layouts, necessitating expensive $O(N)$ memory compaction (e.g., memmove) to fill gaps left by deleted vectors. SIVF, however, leverages its GPU-resident Address Translation Table (ATT) to perform logical deletions. By mapping user IDs directly to physical slots and atomically toggling the corresponding bit in the validity bitmap, SIVF achieves $O(1)$ complexity. This mechanism effectively eliminates the overhead of physical memory reorganization, enabling ultra-low-latency updates essential for real-time streaming applications.

5 Conclusion

Acknowledgment

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation.

References

[1] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library.

[2] Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[3] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[4] Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

A Appendix

A.1 More Experimental Details